

Listes chaînées

1. Idée générale

Dans un *vrai* tableau, les éléments sont contigus en mémoire : *accéder* au *i*-ème élément est *immédiat*, mais **insérer** (ou **supprimer**) un élément « quelque part au milieu » est **impossible** (il *faut* créer un nouveau tableau, plus grand, et recopier les éléments de l'ancien dans le nouveau).

Dans une liste Python, cette **insertion est possible**, mais **coûteuse** en temps (elle oblige à décaler tous les éléments qui suivent).

Une **liste chaînée** adopte une stratégie radicalement différente : les éléments (appelés **nœuds** ou **maillons**) peuvent être « éparpillés » *n'importe où* dans la mémoire vive. Chaque nœud connaît seulement son voisin immédiat grâce à une **référence** (une sorte de lien) vers le nœud suivant.

La liste est ainsi « chaînée » de nœud en nœud, depuis une **tête** jusqu'à un nœud final qui ne pointe vers « rien ».

Ce qui suit la tête est appelé la **queue**.

Exemple : analogie ▼

Il faut imaginer une suite de wagons de train : chaque wagon ne voit que le wagon qui le suit immédiatement.

Le train débute à la locomotive : pour atteindre le 5ème wagon, il faut partir d'elle et avancer wagon par wagon — on ne peut pas « sauter *directement* » au bon endroit.

Remarque : difficulté analogue à celle des arbres (1/2) ▼

Listes chaînées et arbres sont deux structures *naturellement récursives*. Se pose alors la même difficulté, lors de l'implémentation en Python : que désigne vraiment le mot « liste » ? L'ensemble de la structure, ou seulement le nœud de tête ? Et peut-on dire que « ce qui suit la tête » d'une liste est aussi une (sous-)liste ?

Oui, car son premier maillon devient la tête de cette sous-liste. C'est précisément ce qui rend la structure récursive.

Lorsqu'on parle de liste (chaînée), on désigne uniquement le *nœud de tête*. C'est par *convention* qu'on lui donne le rôle de « représentant » de toute la liste. La liste entière n'a pas d'existence propre en mémoire : elle n'est que la succession de nœuds que l'on peut atteindre en partant de la tête.

2. Représentation en Python

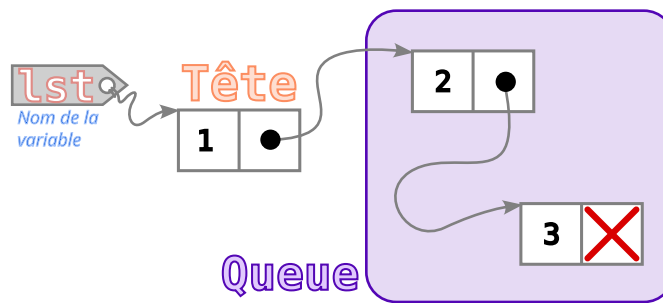
Un nœud se représente naturellement par une classe :

```
class Noeud:
    def __init__(self, valeur):
        self.valeur = valeur
        self.suivant = None          # À la création : pas de suivant
```

Une liste chaînée contenant les valeurs 1, 2, 3 se construit ainsi :

```
lst = Noeud(1)
lst.suivant = Noeud(2)
lst.suivant.suivant = Noeud(3)
```

Ce qui correspond schématiquement à :



! Attention

La liste chaînée est accessible uniquement par sa **tête** : c'est elle que l'on référence par un nom de variable. **Si on perd la référence vers la tête, la liste entière devient *inaccessible*!**

Remarque : variantes ▼

- On peut aussi nommer chaque nœud. Cela donne un code plus verbeux :

```
nd3 = Noeud(3)
nd2 = Noeud(2)
nd2.suivant = nd3
lst = Noeud(1)
lst.suivant = nd2
```

- On peut souhaiter être en mesure de déclarer dès l'instanciation la valeur du nœud *et* le nœud suivant. Le code suivant convient :

```
class Noeud:
    def __init__(self, valeur, suivant=None):
        self.valeur = valeur
        self.suivant = suivant    # Par défaut : pas de suivant
```

À utiliser ainsi :

```
nd3 = Noeud(3)
nd2 = Noeud(2, nd3)
lst = Noeud(1, nd2)
```

Ou directement :

```
lst = Noeud(1, Noeud(2, Noeud(3)))
```

 Remarque : difficulté analogue à celle des arbres (2/2) ▼

Dans le code présenté ici, `lst` désigne uniquement le nœud de tête. Comme indiqué dans [une remarque plus haut](#), on lui donne *par convention* le rôle de « représentant » de toute la liste.

Cela a une conséquence importante : toutes les fonctions qui *modifient la tête* (voir plus bas) doivent renvoyer le **nouveau nœud de tête**, et il faut **impérativement** récupérer cette valeur de retour.

Une alternative consiste à envelopper la liste dans une classe `ListeChaine` contenant un attribut `tete`. La liste aurait alors une existence propre, et les modifications de tête n'imposeraient plus de retour explicite. Ce choix est courant en POO, mais moins pertinent pour comprendre les mécanismes sous-jacents.

3. Opérations fondamentales

3.1. Parcours

Pour parcourir une liste chaînée, on avance de nœud en nœud depuis la tête. On fait « glisser de proche en proche » le maillon pointé par une variable :

```
def afficher(tete):
    courant = tete
    while courant is not None:
        print(courant.valeur)
        courant = courant.suivant      # On « glisse » vers le nœud suivant
```

Utilisation :

```
afficher(lst)      # Affichera 1 puis 2 puis 3
```

3.2. Longueur

Même principe, en incrémentant un compteur à chaque tour de boucle :

```
def longueur(tete):
    compteur = 0
    courant = tete
    while courant is not None:
        compteur += 1
        courant = courant.suivant      # On « glisse » vers le nœud suivant
    return compteur
```

Utilisation :

```
print(longueur(lst))      # Affichera 3
```

3.3. Insertion en tête

Il suffit de créer un nouveau nœud (ou maillon), et d'y rattacher « l'ancienne » liste (par sa tête bien sûr).

```
def inserer_en_tete(valeur, tete):
    nouveau = Noeud(valeur)
    nouveau.suivant = tete
    return nouveau          # Le nouveau nœud devient la nouvelle tête
```

⚠ Piège

`inserer_en_tete` renvoie la nouvelle tête : il faut impérativement récupérer ce retour et le mémoriser (le plus souvent en le réaffectant au nom de la liste d'origine).

```
tete = inserer_en_tete(0, tete)    # ✅ correct
inserer_en_tete(0, tete)          # ❌ la nouvelle tête est perdue !
```

3.4. Suppression du premier nœud de valeur donnée

L'idée est de faire glisser notre « regard » jusqu'au nœud qu'on veut supprimer, **tout en conservant la mémoire du nœud précédent**. Il suffit alors de « sauter » le nœud à supprimer : on rattache sa queue à celle du nœud précédent.

```
def supprimer(valeur, tete):
    # Cas particulier : la tête est le nœud à supprimer
    if tete is None:
        return None          # Liste vide : rien à faire
    if tete.valeur == valeur:
        return tete.suivant   # La nouvelle tête est le nœud suivant

    # Cas général : on cherche le nœud précédant celui à supprimer
    courant = tete
    while courant.suivant is not None:
        if courant.suivant.valeur == valeur:
            courant.suivant = courant.suivant.suivant  # On « saute » le nœud
            return tete
        courant = courant.suivant
    return tete              # Valeur absente : liste inchangée
```

Utilisation :

```
tete = supprimer(2, tete)          # Supprime le premier nœud de valeur 2
```

💬 Remarque : pourquoi chercher le nœud *précédent* ? ▼

Dans une liste *simplement* chaînée, chaque nœud ne connaît que son successeur — pas son prédécesseur. Pour « sauter » un nœud, il faut donc modifier le lien `suivant` de celui qui le précède : `precedent.suivant = precedent.suivant.suivant`. C'est pour cela qu'on s'arrête sur le nœud *avant* celui à supprimer, et non sur le nœud lui-même.

💬 Remarque : liste doublement chaînée ▼

Une **liste doublement chaînée** ajoute à chaque nœud un lien vers le nœud *précédent* en plus du suivant. Dans l'analogie du train, cela signifierait qu'un wagon peut voir *aussi* celui qui le précède. Cela simplifie la suppression (plus besoin de chercher le prédécesseur) et permet de parcourir la liste dans les deux sens — au prix d'une légère augmentation de la mémoire utilisée.

3.5. Insertion à une position donnée (au-delà de la tête)

Insérer un nœud à une position donnée repose sur le principe suivant : une fois le nœud précédant la position atteint, deux redirections suffisent — sans toucher aux autres nœuds :

```
nouveau.suivant = courant.suivant
courant.suivant = nouveau
```

⚠ **L'ordre des deux lignes est crucial !**

Si on inversait, `courant.suivant` serait écrasé avant d'avoir été sauvegardé, et la suite de la liste serait perdue.

Voici le code complet.

```
def inserer(valeur, position, tete):
    # Précautions nécessaires
    assert position > 0, "Pour insérer en tête, utiliser inserer_en_tete()"
    assert tete is not None, "Impossible d'insérer dans une liste vide"
    # Cas général : on cherche la position où insérer
    courant = tete
    pos_courant = 0
    # Avec la condition ci-dessous, si la position spécifiée pour l'insertion
    # est supérieure à la longueur de la liste, la valeur sera insérée en queue
    while courant.suivant is not None and pos_courant < position - 1:
        courant = courant.suivant          # On « glisse » vers le nœud suivant
        pos_courant += 1                    # On incrémente le compteur de nœuds
    # On a enfin atteint la position d'insertion
    nouveau = Noeud(valeur)
    nouveau.suivant = courant.suivant      # On rattache la queue au nouveau nœud
    courant.suivant = nouveau              # On accroche le nouveau nœud après courant
```

💬 **Remarque : pourquoi un `assert` dans ce code ? ▼**

`insérer_en_tete` renvoie la nouvelle tête, ce qui impose une réaffectation à la variable. `insérer` modifie la liste en place et **ne renvoie rien**. Utiliser `insérer` en position 0 ne planterait pas forcément, mais produirait un résultat silencieusement incorrect : le nœud inséré ne deviendrait jamais la vraie tête. L'`assert` rend cette erreur immédiatement visible.

💬 **Remarque : position d'insertion trop grande ? ▼**

Si la position où l'insertion est demandée dépasse la longueur de la liste, il y a deux approches possibles :

- refuser et afficher un message (ou mieux : lever une exception avec un `assert`) ;
- insérer silencieusement en queue. C'est le choix fait dans le code ci-dessus.

4. Complexité des opérations

Opération	Liste chaînée	Tableau (liste Python)
Accès au <code>i</code> -ème élément	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Insertion / suppression en tête	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Insertion / suppression en milieu (nœud connu)	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Insertion / suppression en milieu (position par indice)	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Parcours complet	$\mathcal{O}(n)$	$\mathcal{O}(n)$

La liste chaînée est donc avantageuse lorsque les **insertions et suppressions sont fréquentes** et qu'on dispose déjà d'une référence vers le nœud concerné. Elle est désavantageuse pour l'**accès direct par indice**.

5. Points d'attention

Erreurs classiques

- **Oublier de vérifier `None`** avant d'accéder à `nœud.valeur` ou `nœud.suivant` → erreur `AttributeError` si le nœud n'existe pas.
- **Ne pas récupérer la nouvelle tête** après une insertion ou suppression en tête → la variable `tete` pointe toujours sur l'ancienne tête.
- **Inverser l'ordre des redirections** lors d'une insertion ou suppression → perte de la suite de la liste.
- **Confondre liste chaînée et liste Python** : la liste Python offre l'accès par indice en $\mathcal{O}(1)$; la liste chaînée non.

6. Liens avec d'autres notions

- **Tableau** : structure complémentaire — accès direct efficace, insertion coûteuse. Le choix entre les deux dépend des opérations dominantes du problème.
- **Piles et files** : peuvent s'implémenter efficacement avec une liste chaînée (insertion et suppression en tête en $\mathcal{O}(1)$).
- **Arbres** : généralisation de la liste chaînée où chaque nœud peut avoir plusieurs successeurs.
- **Complexité** : la notation $\mathcal{O}$ permet de comparer objectivement les structures indépendamment de la machine utilisée.